Day 15: Classes & Objects in Embedded

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Class Basics in C++

A `class` groups data (member variables) and operations (member functions) that belong together. In embedded, classes model hardware peripherals, protocol stacks, and application components:

```
class UartDriver {
public:
    bool init(const struct device *dev);
    bool send(const uint8_t *data, size_t len);
    bool recv(uint8_t *buf, size_t len);
private:
    const struct device *dev_;
    bool initialized_;
};
```

Access Specifiers

- `public`: accessible from anywhere — the class interface
- `private`: accessible only from within the class — implementation details
- `protected`: accessible from derived classes — use sparingly in embedded

Rule: Data should almost always be `private`. Expose only the operations needed by callers.

Member Functions

Member functions have access to all member variables via the implicit `this` pointer:

```
bool Led::toggle() {
    state_ = !state_;                          // Accesses member variable
    return gpio_pin_set_dt(spec_, state_);     // Uses member pointer
}
```

Embedded OOP Rules

Embedded C++ should follow these constraints:

1. **No exceptions** (`-fno-exceptions`) — stack unwinding wastes code size and time
2. **No RTTI** (`-fno-rtti`) — `dynamic_cast` , `typeid` add overhead
3. **Minimize virtual functions** — each class with virtuals gets a 4-byte vtable pointer
4. **Prefer `static` member functions** for operations that don't need `this`
5. **Check object sizes** with `static_assert(sizeof(MyClass) <= 32)`

No RTTI, No Exceptions

Zephyr compiles C++ with `-fno-exceptions -fno-rtti` by default:

- `throw` / `catch` → use return codes or `Result` instead
- `dynamic_cast` → use a type tag or interface check instead
- `typeid` → use a `type()` virtual function instead

Object Size Awareness

Every class instance consumes RAM. On nRF52840 with only 256KB RAM, class sizes matter:

```
class Led { /* ... */ };
static_assert(sizeof(Led) <= 16, "Led too large");  // Catch unexpected bloat

// Array of 8 sensors — must fit in RAM
static Sensor sensors[8];
static_assert(sizeof(sensors) < 256, "Sensor array too large");
```

Practice Tasks

1. Create a `PwmLed` class that wraps Zephyr PWM API with `set_brightness(0-100)` method
2. Add `static_assert` to verify your class sizes — intentionally add a large member and observe the failure
3. Try compiling with a `try/catch` block — observe the compile error with `-fno-exceptions`
4. Write a `Button` class that debounces input and fires a callback on valid press

---

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day15
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach   # RTT viewer (requires J-Link)
```

Resources

- Zephyr Docs
- nRF52840 Product Specification
- Nordic DevZone
- Source: src/day15/main.cpp